Multimodal apps with AI Foundry and .NET MAUI

Microsoft



David Ortinau
Principal Product Manager
david.ortinau@microsoft.com

Agenda

01 The Multimodal Shift
02 Azure AI Foundry
03 Multimodal Patterns
04 Architecture with .NET MAUI
05 UX Design Principles
06 Next Steps

01 The Multimodal Shift

The Multimodal Shift
The (d)evolution of UI

Text
Text is slow, precise, and limiting. Forms can be misinterpreted. Usability is a challenge.

Voice
Voice is fast, and natural. Voice carries additional context and is expressive.

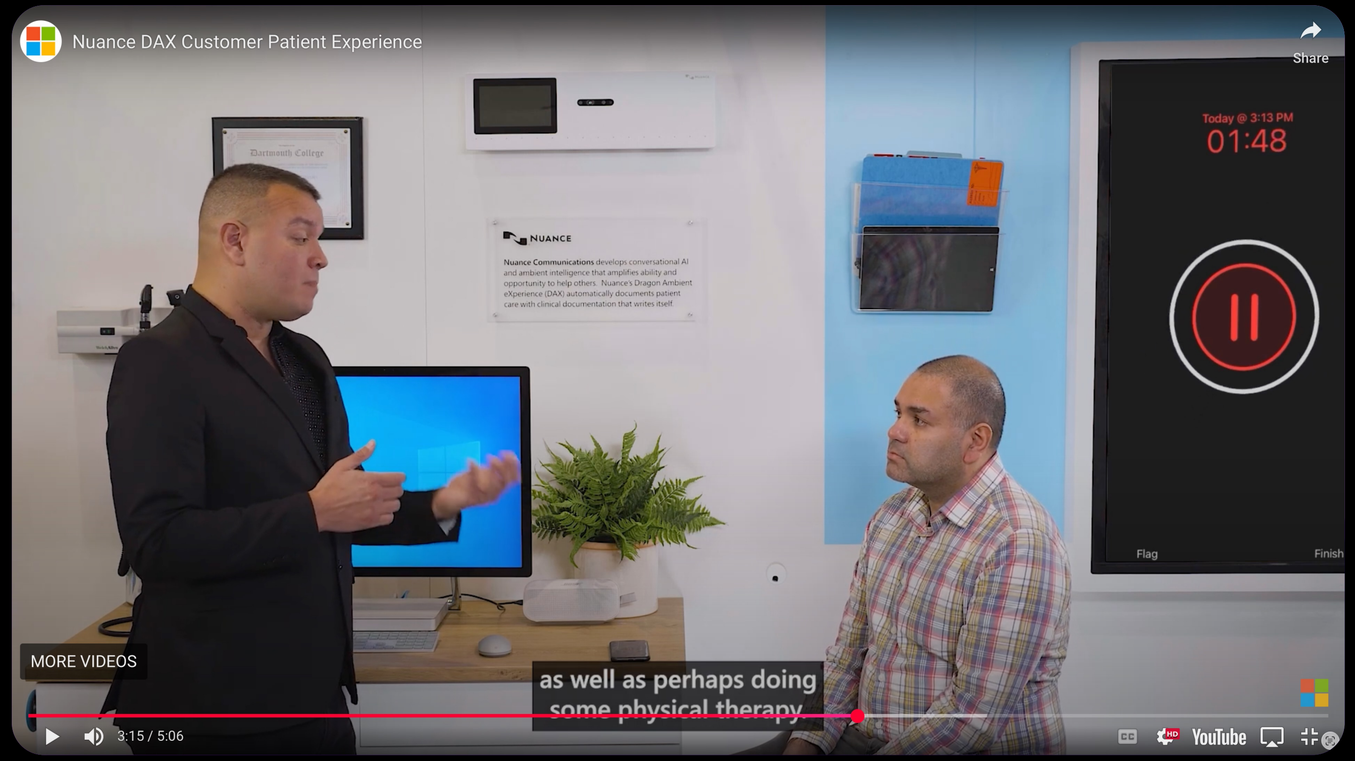
Vision
A picture is worth a thousand words.

"Multimodal experiences blend different input and output modes to create richer, more natural interactions.

Microsoft + Nuance DAX

Ambient AI in the Exam Room

In many hospital settings, doctors are drowning in paperwork. What if the very conversation they have with you could **become the record**, without lifting a finger? That's what ambient AI does. In one demo, Nuance's DAX listens while the patient and physician talk, and produces structured, ready-to-review notes — combining speech, language, context, and domain awareness. It's a powerful example of multimodal intelligence in action.



83%

Say their physician is more personable and conversational

81%

Say their physician is more focused

75%

Say their physician spends less time on the computer

02 Azure AI Foundry

Azure AI Foundry Overview
Key Capabilities for Developers



01

PaaS

A unified Platform-as-a-Service (PaaS) for enterprise AI operations, model development, and application lifecycle management.

02

Models

Build, evaluate, and deploy models from a broad catalog of AI providers.

03

Apps & Agents

Complete infrastructure to compose multimodal applications and intelligent agents, so you can focus on solutions—not plumbing.

04

Security & Trust

Built-in security, compliance, and governance guardrails protect both developers and end-users from the start.

05

Unified

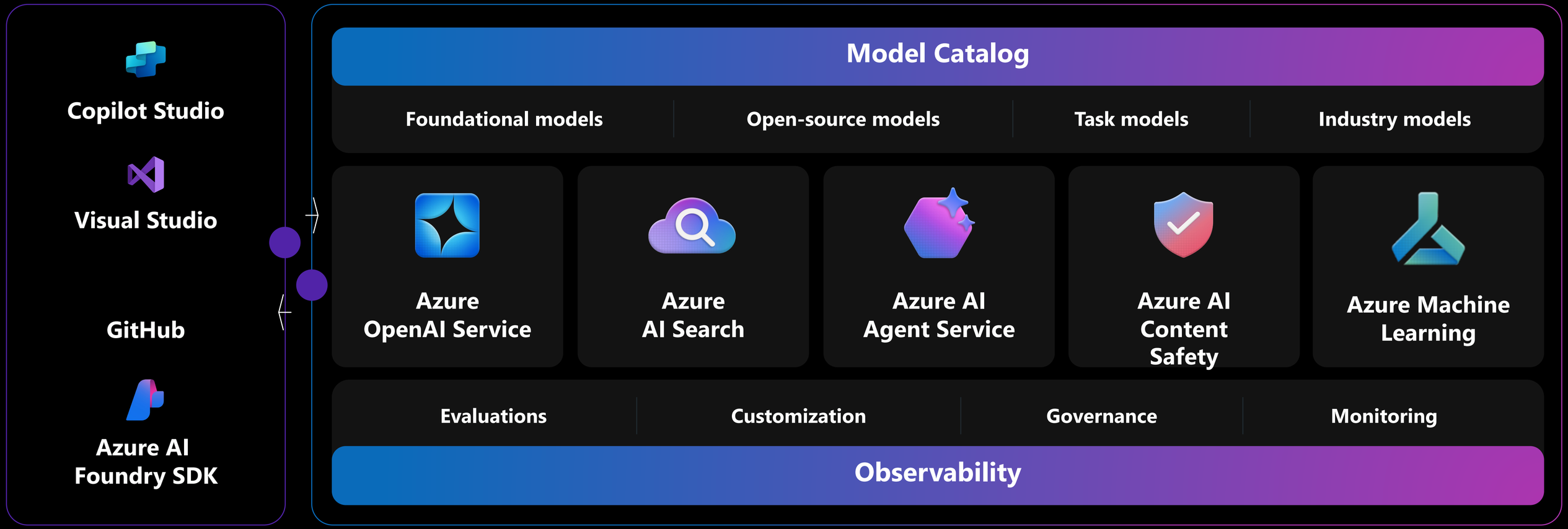
Unified with Visual Studio, VS Code, GitHub, and Azure—bringing the full Microsoft cloud into familiar developer workflows.

06

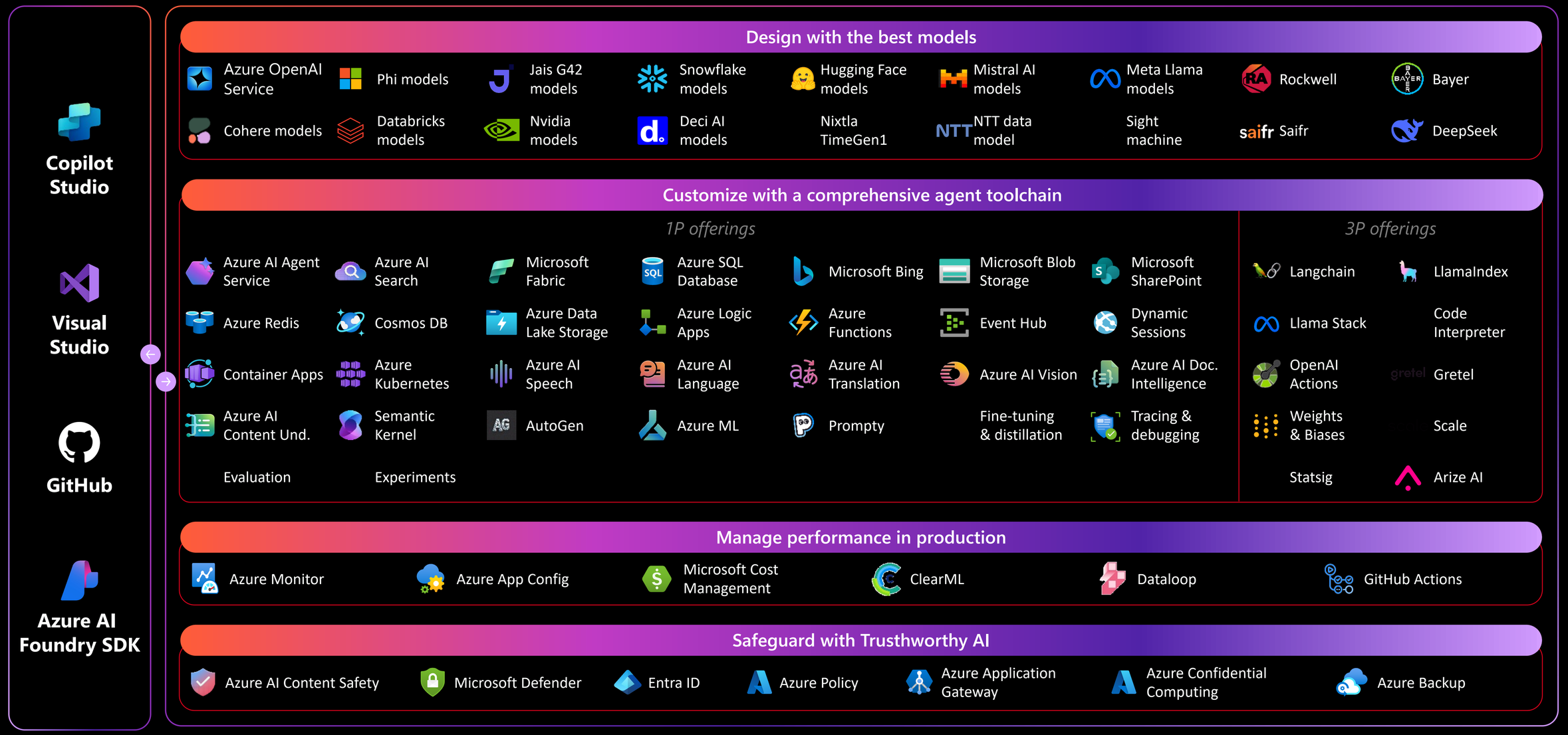
Scalability

Abstracts the complexity of running AI at scale—take ideas from prototype to global deployment and monitoring in just a few clicks.

Azure AI Foundry



Azure AI Foundry agent ecosystem



Design intelligent apps with Azure AI services

Leverage out-of-the-box and customizable APIs and multimodal models

Azure OpenAI Service

- Access to powerful AI models
- Scalable development
- Compliance & security
- Integration with other Azure Services

Azure AI Search

- AI enrichment & semantic ranking
- Generative AI content creation
- Vector search for data organization

Azure AI Speech

- Speech to text (including the Whisper model on Azure OpenAI Service)
- Text to speech
- Speech translation
- Speaker recognition

Azure AI Vision

- Image and face analysis
- Custom model training
- Face detection and recognition
- Document text extraction

Azure AI Content Safety

- AI-driven content moderation for enhanced safety
- Customize safety thresholds for diverse user types
- Detect and prevent Jailbreak Risk from XPIA attacks

Azure AI Document Intelligence

- Automated documentation generation
- Documentation quality analysis
- Interactive documentation experiences
- Natural language understanding for documentation

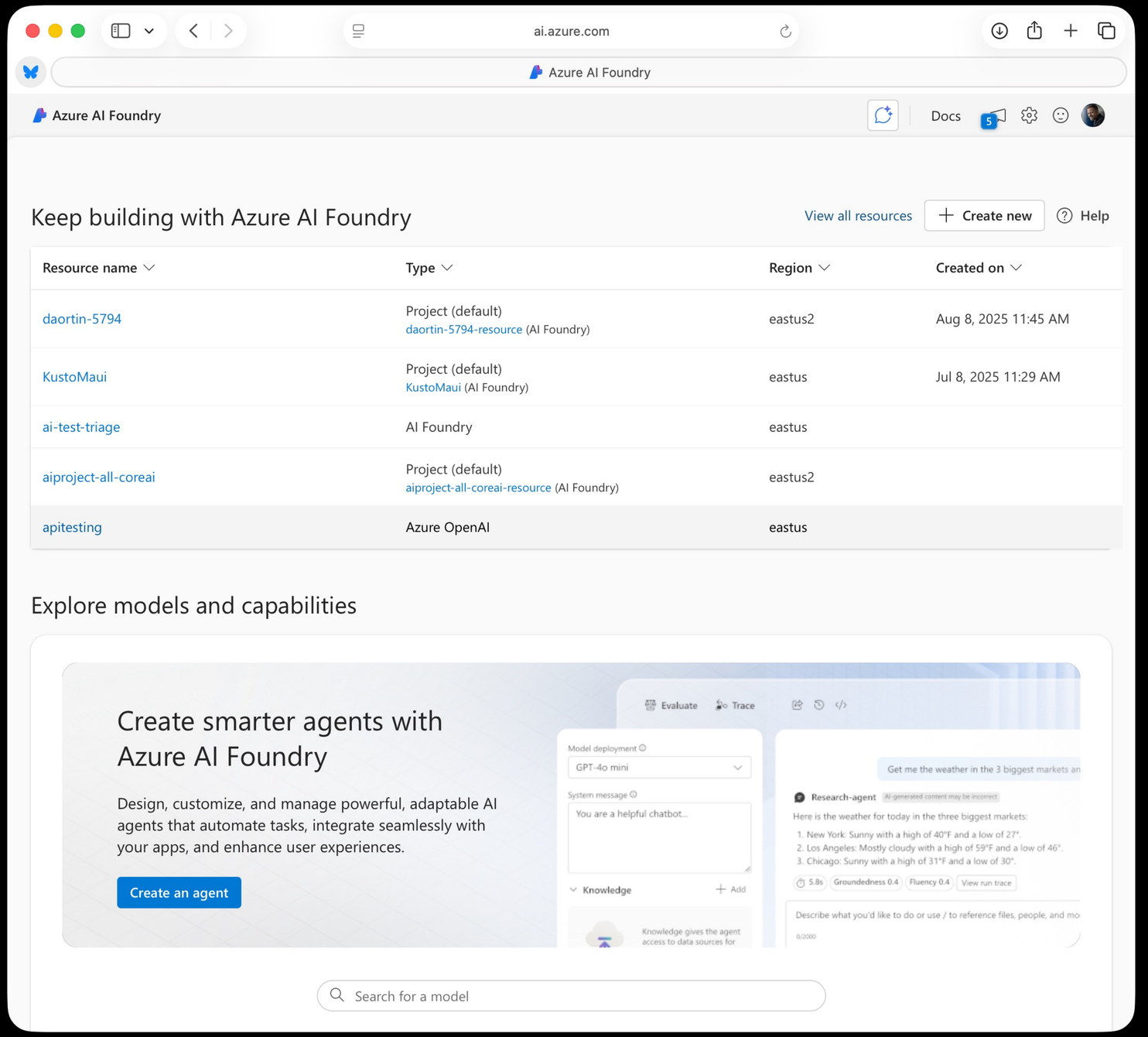
Azure AI Language

- Task-optimized AI models for text analytics
- Custom industry-specific AI for healthcare
- Custom, industry-specific models

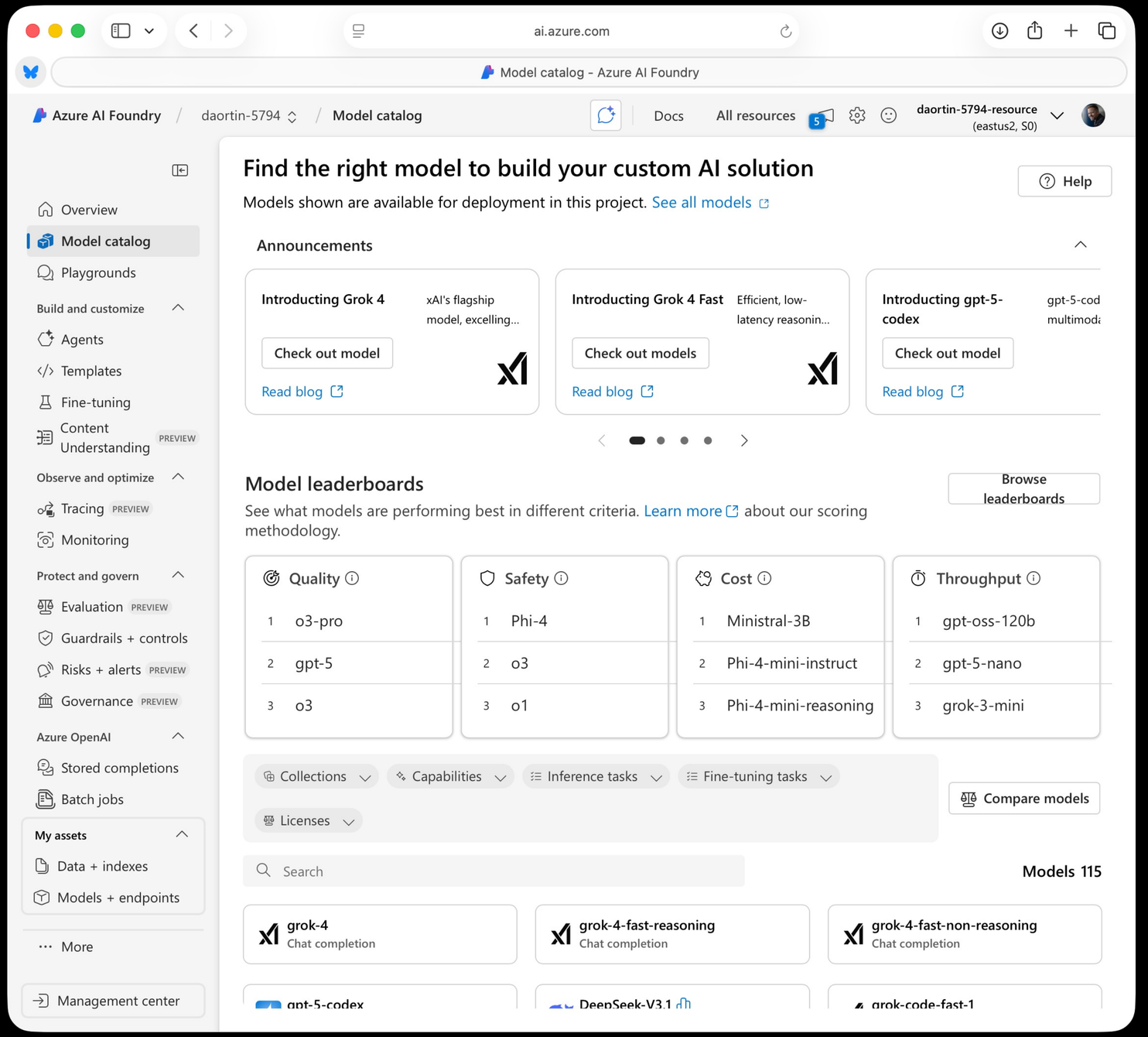
Azure AI Translator

- Multilingual text and speech translation
- Synchronous and asynchronous translation request support
- Native translation of documents and manuals

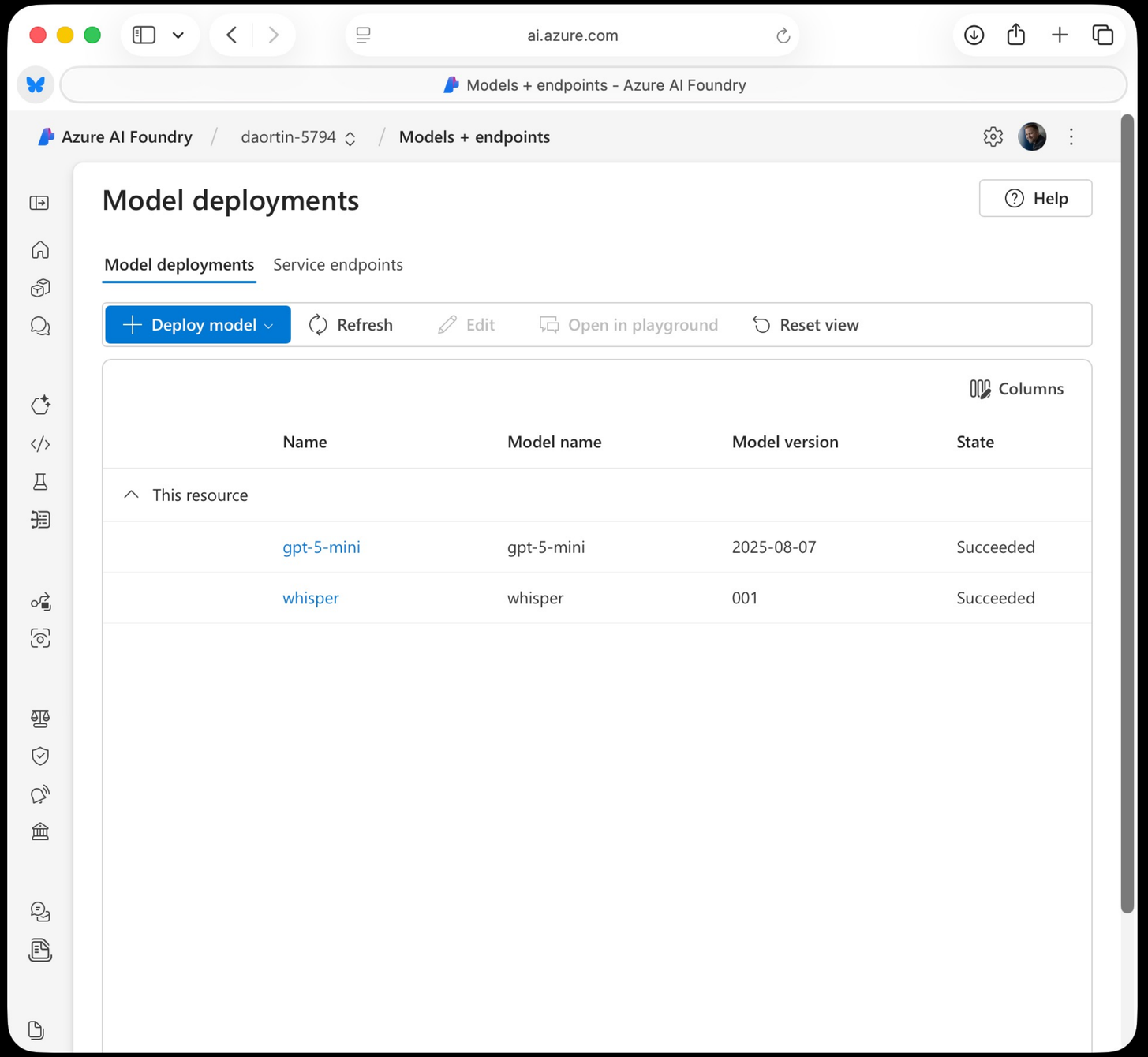
Azure AI Foundry
The Portal



Azure AI Foundry
Model catalog

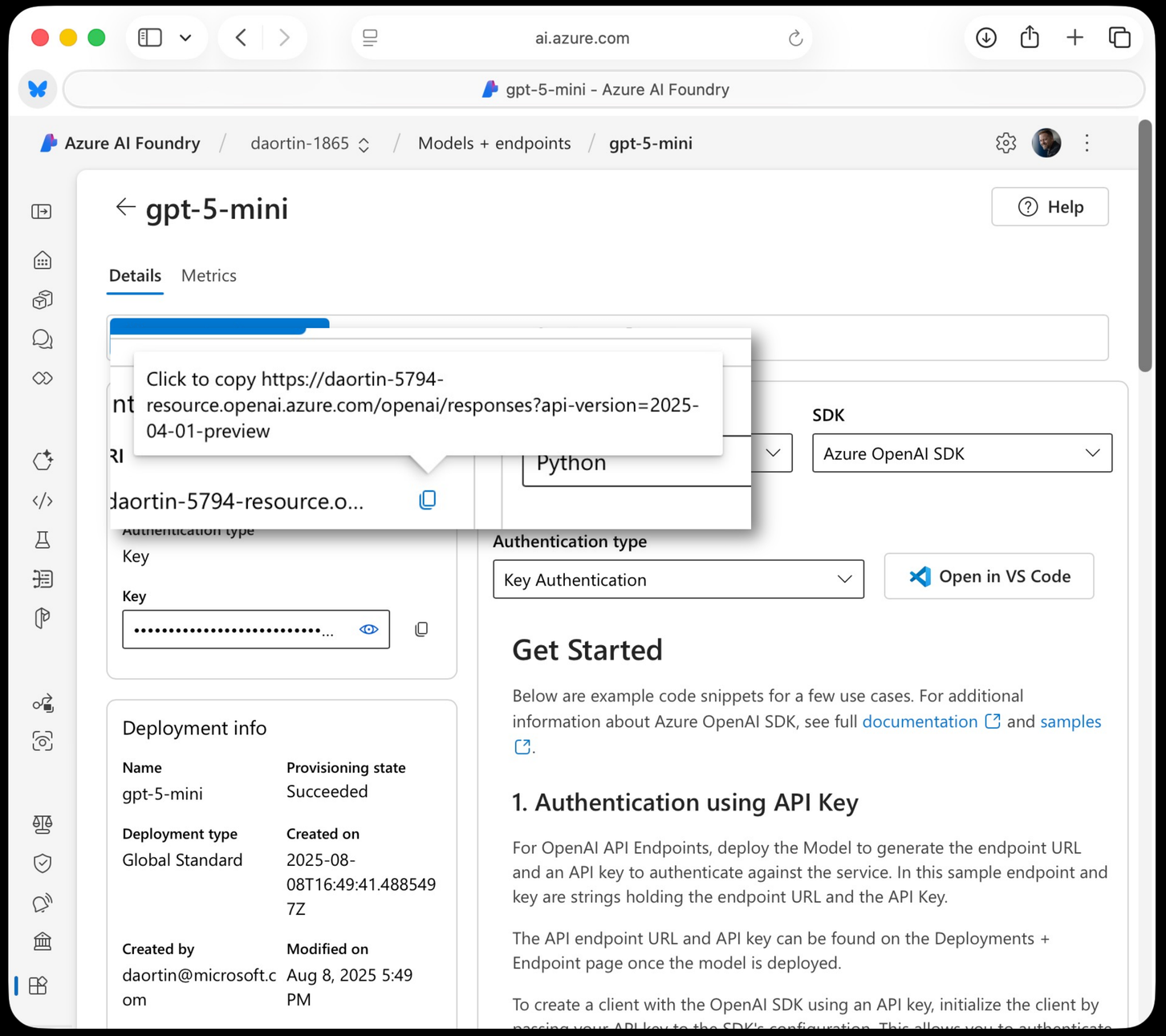


Azure AI Foundry
My assets | Models

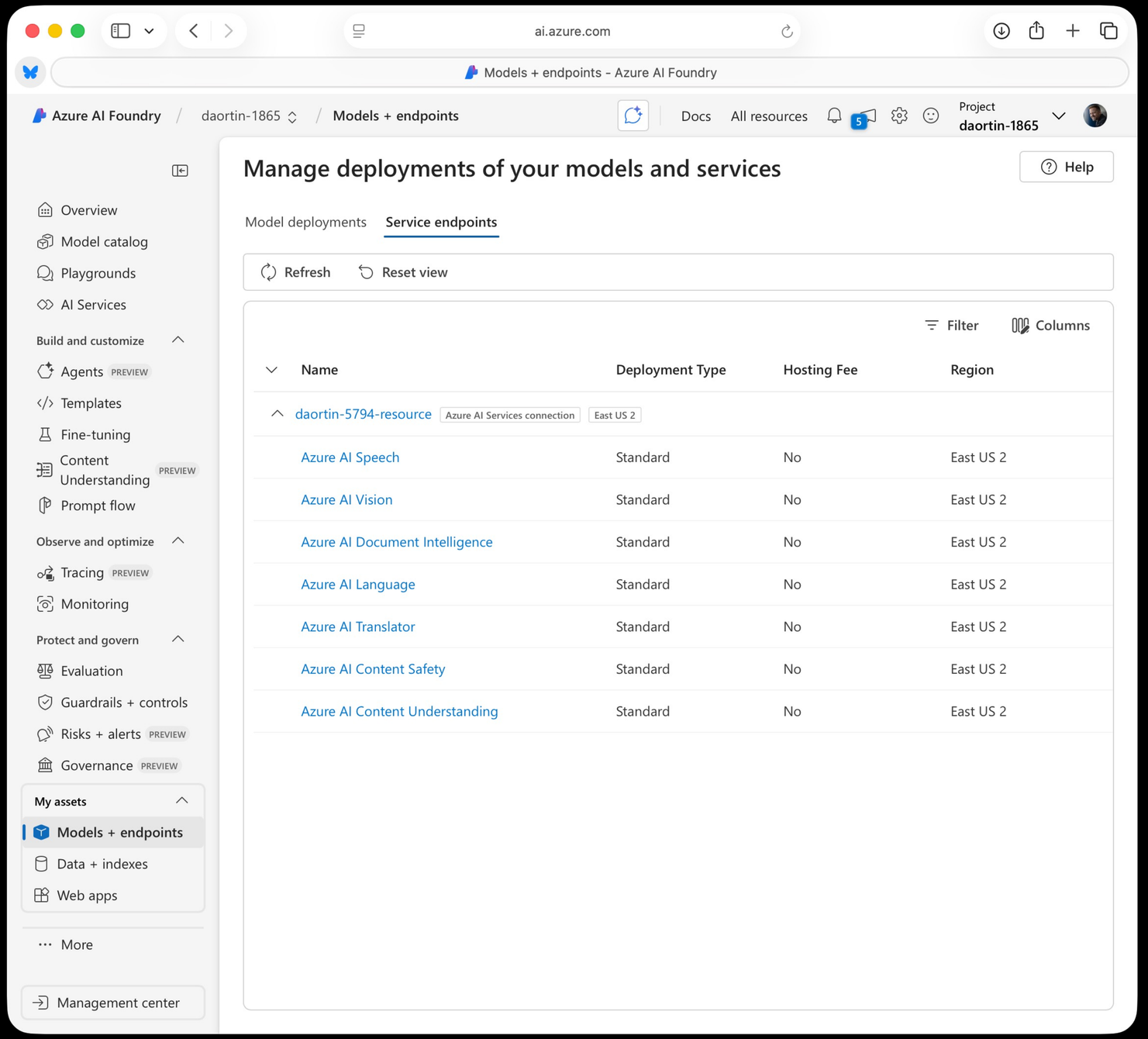


Azure AI Foundry

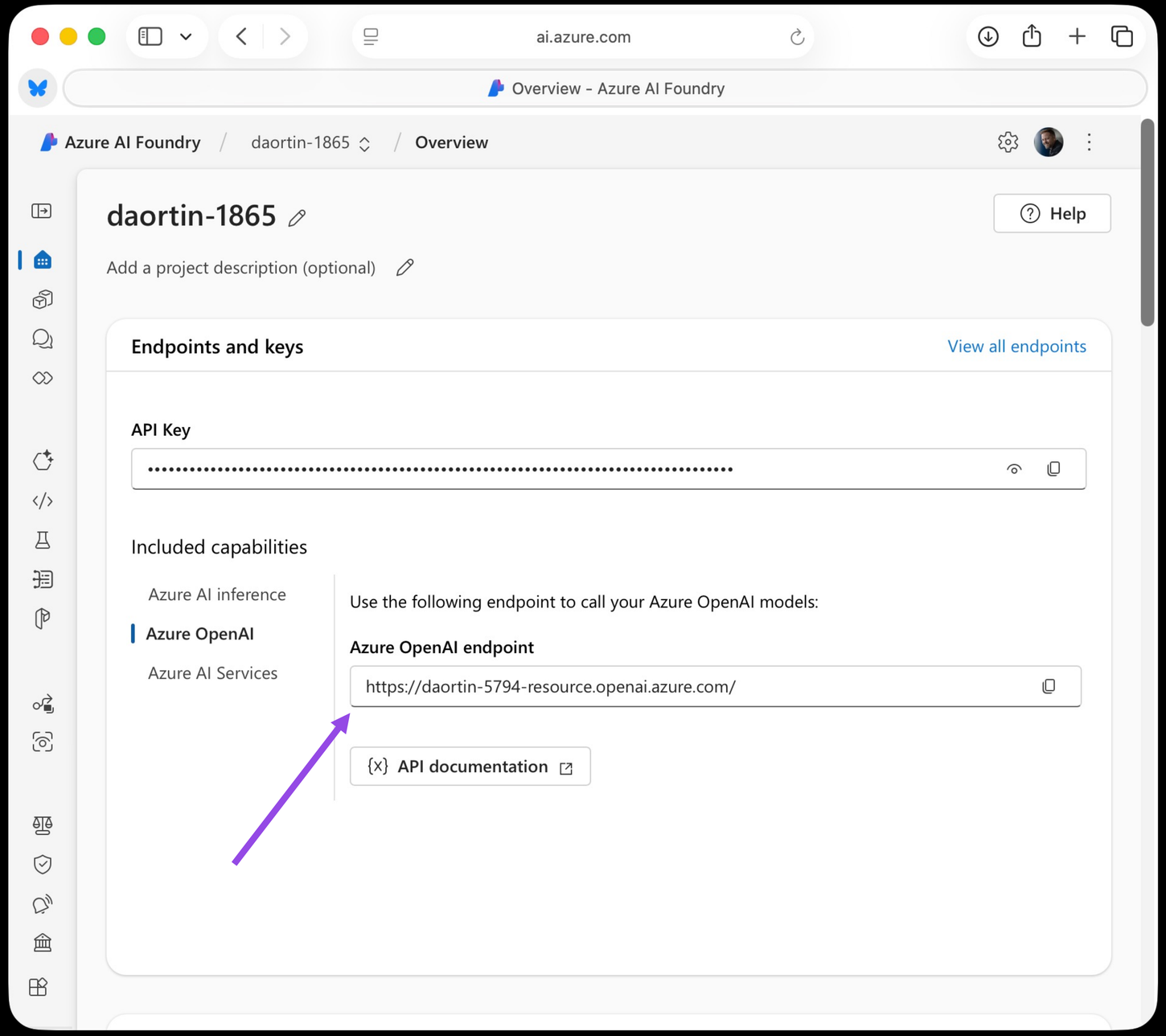
My assets | Model Details



Azure AI Foundry
My assets | Services



Azure AI Foundry
Overview



03 AI with .NET MAUI

Fundamentals

.NET MAUI Packages

01

Microsoft.Extensions.AI

The abstractions and DI glue for coding against. e.g. IChatClient, chat/message models.

02

Microsoft.Extensions.AI.OpenAI

The adapter that plugs the vendor SDK into the Microsoft.Extensions.AI abstractions. Connects to OpenAI or Azure OpenAI in AI Foundry.

03

Azure.AI.OpenAI

The official Azure SDK for Azure OpenAI/Foundry models. Provides GetChatClient(), GetAudioClient() etc.

04

Plugin.Maui.Audio

A cross-platform library for working with audio recording and playback.

05

CommunityToolkit.Maui

Great for a wide variety of things, and here specifically for Voice-to-text.

06

M...Extensions.AI.AzureAIInference

The generic client for Azure AI Foundry that works with OpenAI, Hugging Face, Phi, and custom models.

A Basic Setup

```
var builder = MauiApp.CreateBuilder();
builder
  .UseMauiApp<App>()
  .UseMauiCommunityToolkit()
  .Services.AddChatClient(
        new OpenAIClient(openAiApiKey)
                .AsChatClient("gpt-5-mini"));
```

A Basic Usage

```
public class MyService
{
    public MyService(IChatClient chatClient)
    {
        ChatClient = chatClient;
    }

    public async Task<Sentiment> GetSentiment(string review)
    {
        var response = await chatClient
                        .GetResponseAsync<Sentiment>($"What's the sentiment of this review? {review}");

      return response.Result;
    }
}
```

A More Advanced Setup

```
_client = new AzureOpenAIClient(new Uri(endpoint), new System.ClientModel.ApiKeyCredential(apiKey));
_chatClient = _azureOpenAIClient
                .GetChatClient("gpt-5-mini")
                .AsIChatClient();

_chatClient = new ChatClientBuilder(_chatClient)
  .ConfigureOptions(options =>
  {
    options.Tools ??= [];
    options.Tools.Add(AIFunctionFactory.Create(_locationTools.IsNearby));
  })
  .UseFunctionInvocation()
  .Build();
```

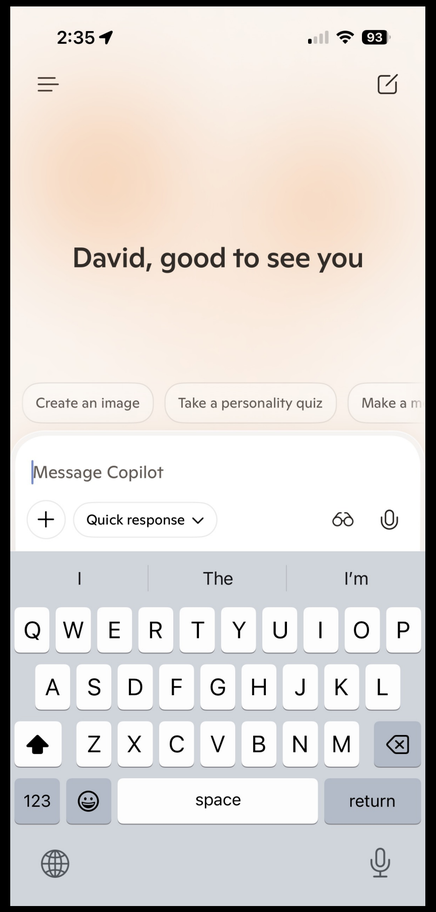
Usage

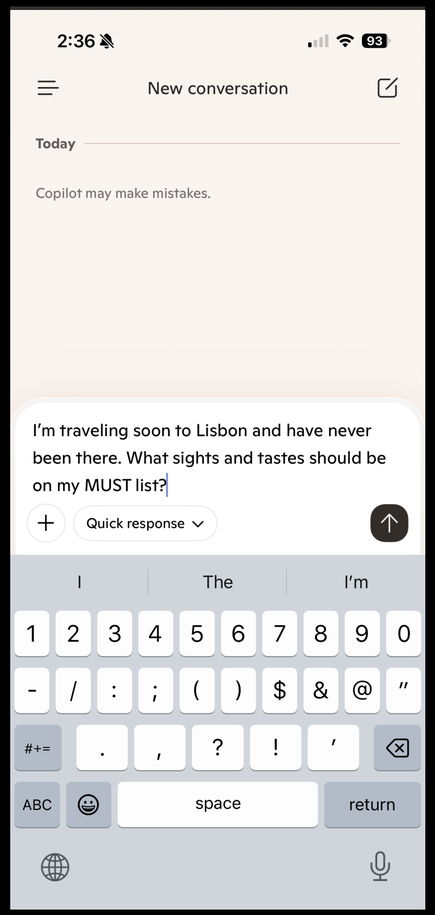
```
var apiResponse = await chatClient.GetResponseAsync<PriorityTaskResult>(sb.ToString());
if (apiResponse?.Result != null)
{
  // Update personalized greeting if it exists
  if (!string.IsNullOrEmpty(apiResponse.Result.PersonalizedGreeting))
  {
    PersonalizedGreeting = apiResponse.Result.PersonalizedGreeting;
  }

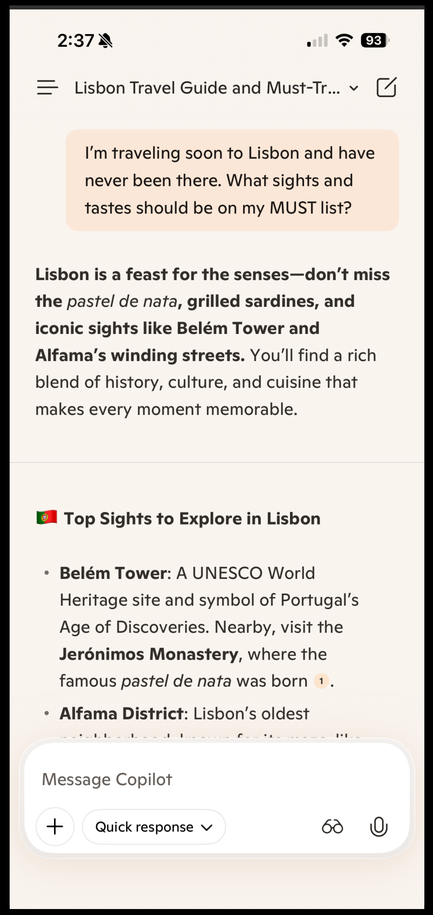
...
}
```

04 Multimodal Patterns

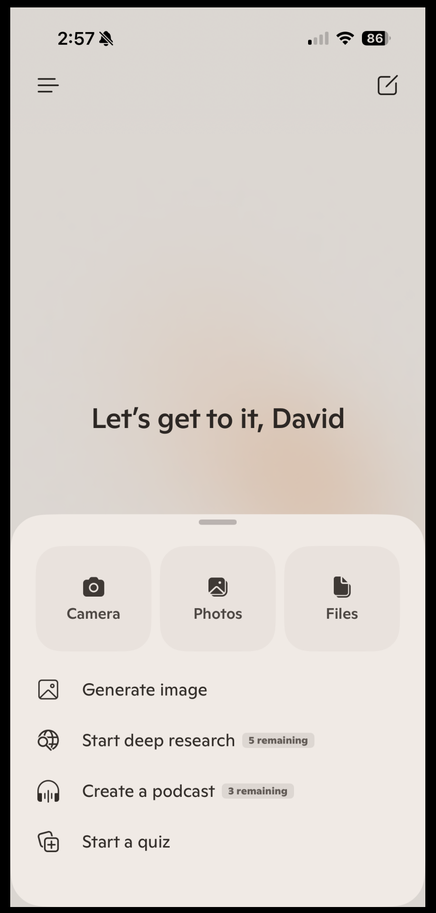
Text
Unimodal

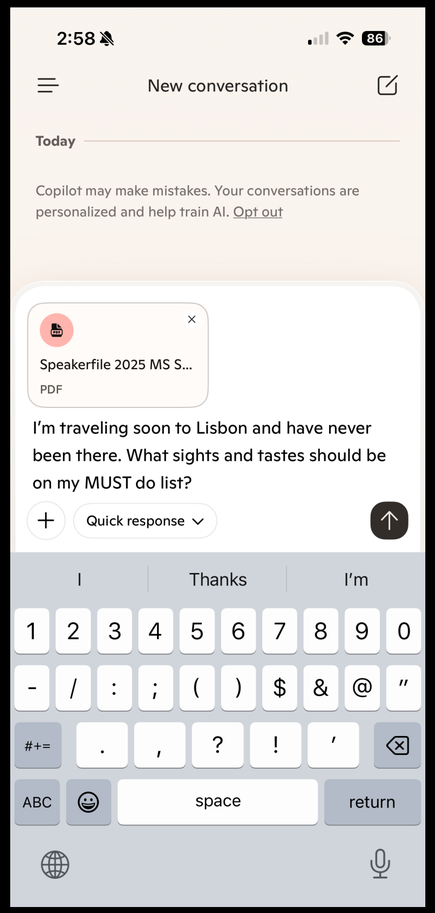




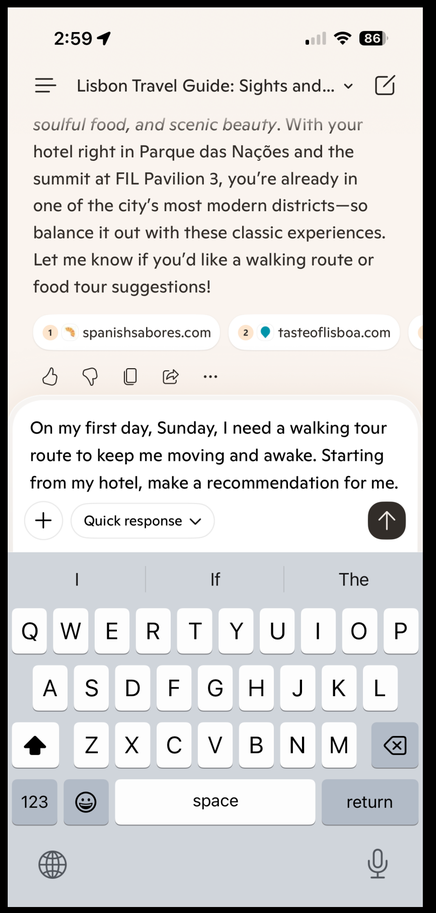


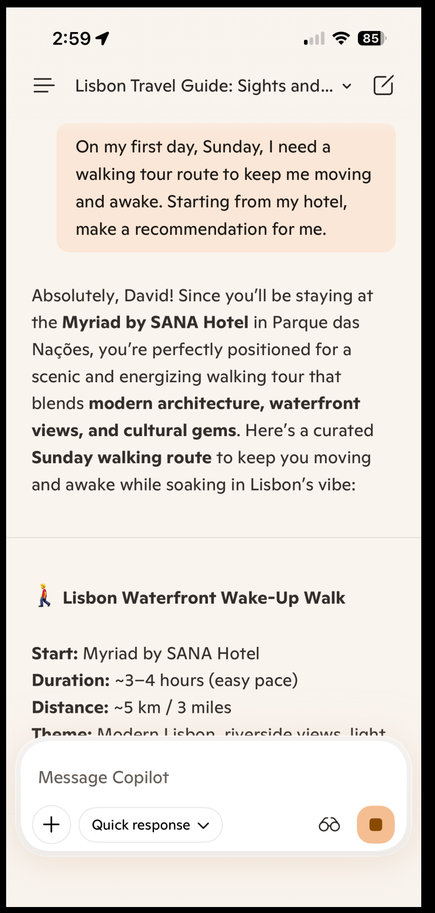
Text + Context
Multimodal



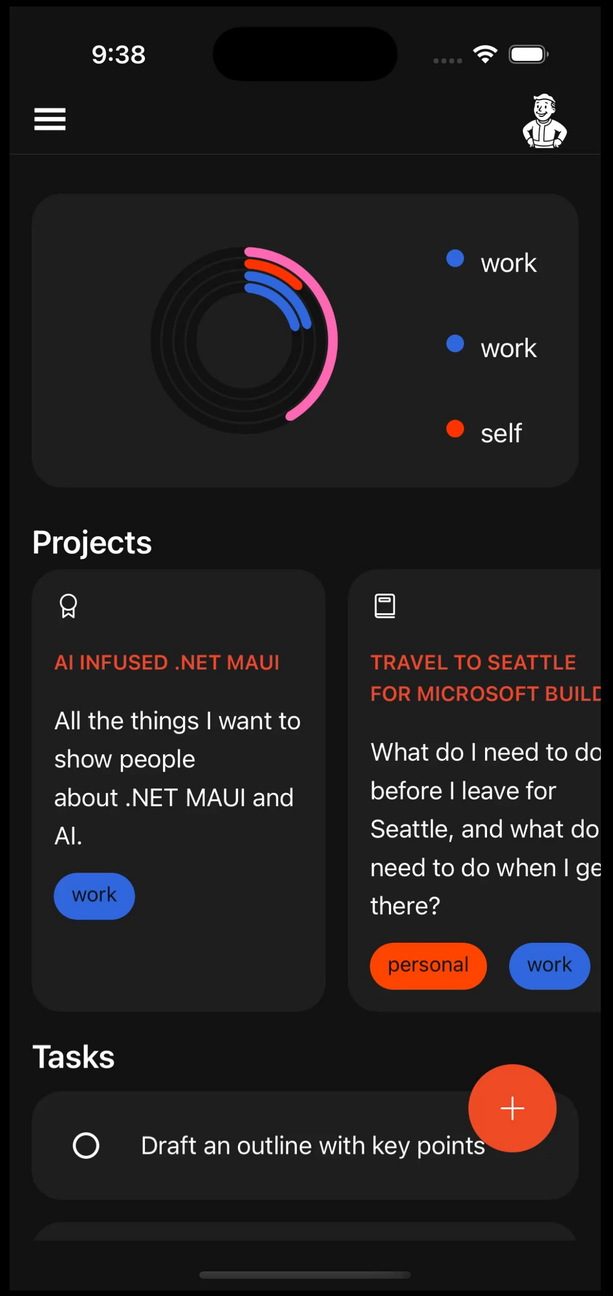








04.01 Text demo



"In the workplace, voice isn't about novelty. It's about speed, accessibility, multitasking, and flow.

Katie Derthick
Microsoft Outlook Research

Voice
Multimodal





Apps with voice commands saw up to 25% increase in retention rates

71% of users prefer voice commands over traditional input

20% higher adherence observed in health and fitness apps

25% higher engagement in consumer apps

March 2025 Grady Andersen & MoldStud Research Team report
Feb 2025 Forrester report
Journal of mobile health

How to Voice
Interaction Modes

Voice as Input

Commands, transcription hands-free, dictation

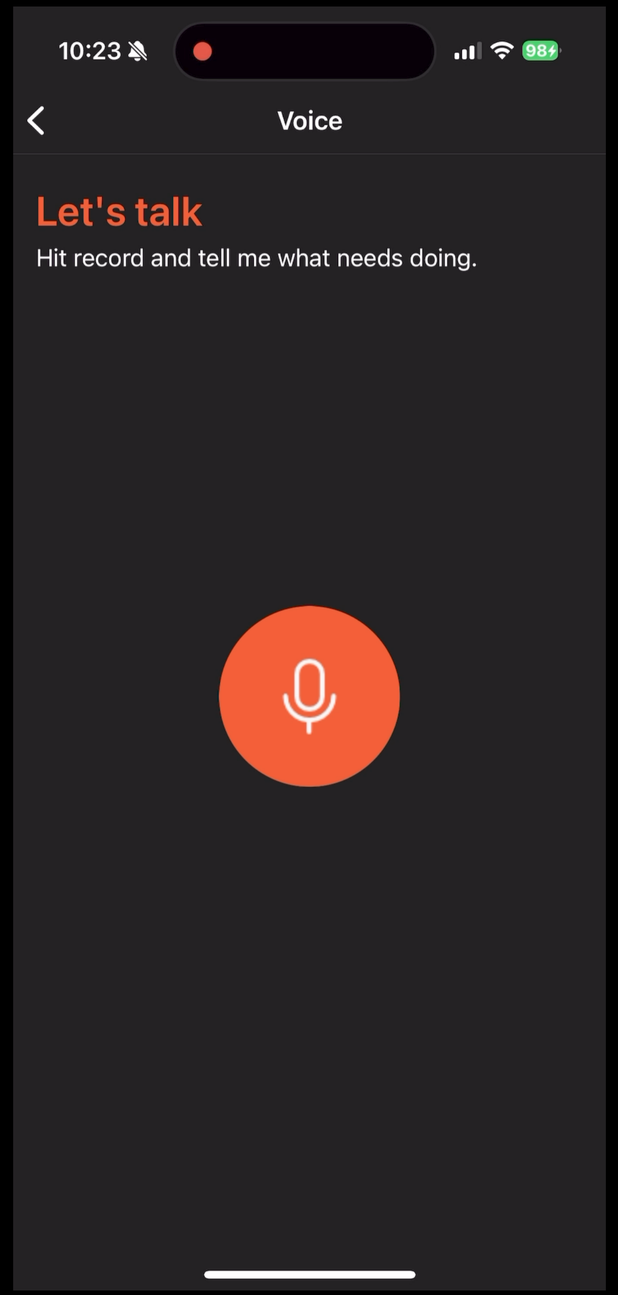
Voice as Output

Read aloud, outlook play my email, maps voice navigation

Voice as conversation

Copilot, ChatGPT, Alexa, assistants

04.02 Voice demo



Capturing Voice Input – Plugin.Maui.Audio

```
using Plugin.Maui.Audio;

public AudioService(IAudioManager audioManager)
{
    _recorder = audioManager.CreateRecorder();
    RecordedFilePath = Path.Combine(FileSystem.Current.CacheDirectory, "dictation.wav");
}

public async Task StartRecordingAsync(CancellationToken ct)
{
    if (File.Exists(RecordedFilePath)
        File.Delete(RecordedFilePath);
    await _recorder.StartAsync(RecordedFilePath);
}

public async Task StopRecordingAsync()
{
    await _recorder.StopAsync();
}
```

https://github.com/jfversluis/Plugin.Maui.Audio

FoundryTranscriptionService.cs

```
public async Task<string> TranscribeAsync(string path, CancellationToken ct)
{
  await using var stream = File.OpenRead(path);

  // Get the Azure OpenAI client from the chat client service
  var azureClient = _chatClientService.GetAzureOpenAIClient();

  // Azure OpenAI (Foundry) audio transcription call
  var result = await azureClient
    .GetAudioClient(_deploymentName)
    .TranscribeAudioAsync(stream, "file.wav", cancellationToken: ct);

  return result.Value.Text.Trim();
}
```

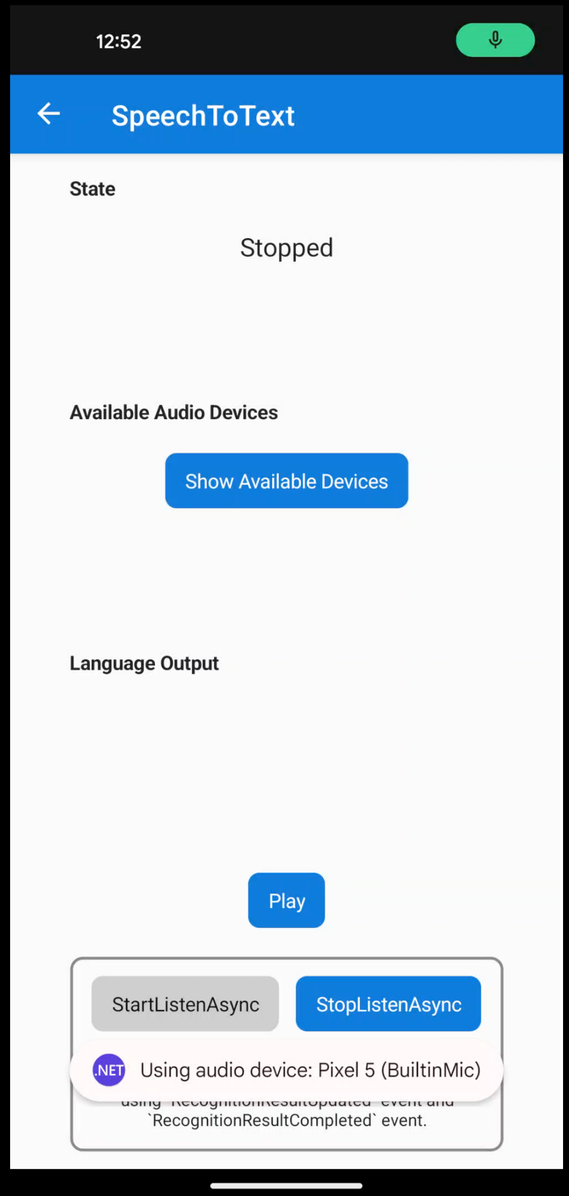
Capturing Voice Input - SpeechToText

```
async Task StartListening(CancellationToken cancellationToken)
{
    // check permissions

    speechToText.RecognitionResultUpdated += OnRecognitionTextUpdated;
    speechToText.RecognitionResultCompleted += OnRecognitionTextCompleted;
    await speechToText.StartListenAsync(new SpeechToTextOptions {
                                            Culture = CultureInfo.CurrentCulture,
                                            ShouldReportPartialResults = true },
                                        CancellationToken.None);
}

async Task StopListening(CancellationToken cancellationToken)
{
    await speechToText.StopListenAsync(CancellationToken.None);
    speechToText.RecognitionResultUpdated -= OnRecognitionTextUpdated;
    speechToText.RecognitionResultCompleted -= OnRecognitionTextCompleted;
}

void OnRecognitionTextUpdated(object? sender, SpeechToTextRecognitionResultUpdatedEventArgs args)
{
    RecognitionText = args.RecognitionResult;
}
```



https://learn.microsoft.com/dotnet/communitytoolkit/maui/essentials/speech-to-text

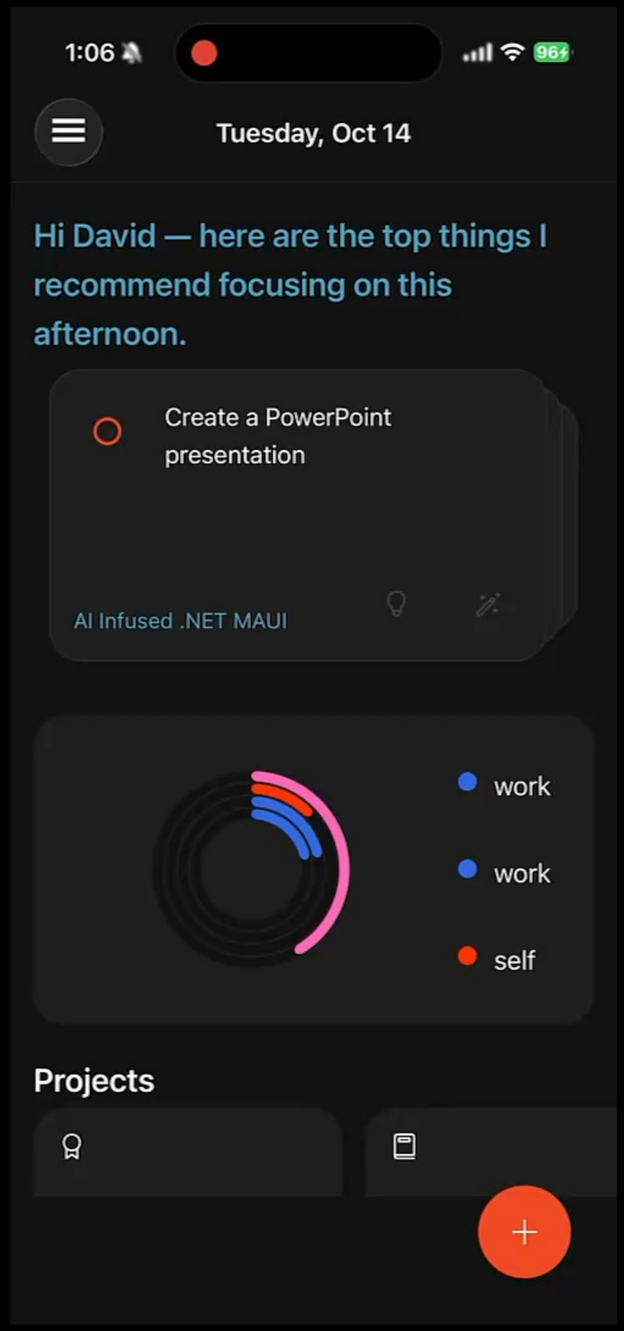
Capturing Voice Input - SpeechToText

```
async Task StartListening(CancellationToken cancellationToken)
{
    // check permissions

    speechToText.RecognitionResultUpdated += OnRecognitionTextUpdated;
    speechToText.RecognitionResultCompleted += OnRecognitionTextCompleted;
    await speechToText.StartListenAsync(new SpeechToTextOptions {
                                            Culture = CultureInfo.CurrentCulture,
                                            ShouldReportPartialResults = true },
                                        CancellationToken.None);
}

async Task StopListening(CancellationToken cancellationToken)
{
    await speechToText.StopListenAsync(CancellationToken.None);
    speechToText.RecognitionResultUpdated -= OnRecognitionTextUpdated;
    speechToText.RecognitionResultCompleted -= OnRecognitionTextCompleted;
}

void OnRecognitionTextUpdated(object? sender, SpeechToTextRecognitionResultUpdatedEventArgs args)
{
    RecognitionText = args.RecognitionResult;
}
```



https://learn.microsoft.com/dotnet/communitytoolkit/maui/essentials/speech-to-text

Voice Output
Implementation options

Text-to-Speech

On-device

On device processing is fast, but voice options are limited and unnatural. Do you really want to live in the past?

AI Text-to-Speech

On-device, Cloud

OpenAI and Microsoft and others offer voice models. Options are limited, and potentially more and more unnatural outside of English.

On-device models are available and may work with Foundry local, ONNX, or Ollama.

AI Live

Cloud

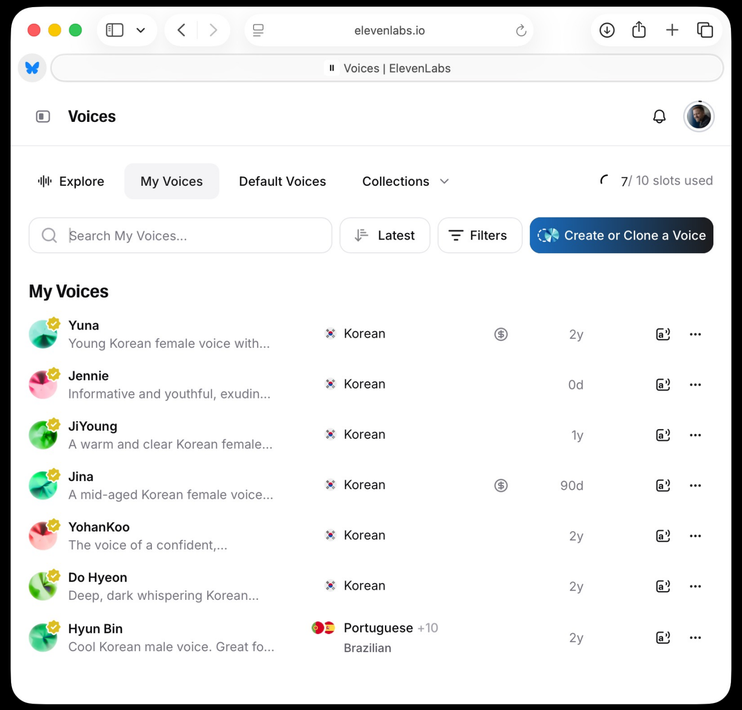
Conversational voice is offered by OpenAI, Azure, and Foundry. The UX is currently locked to voice-to-voice.

AI Specialized

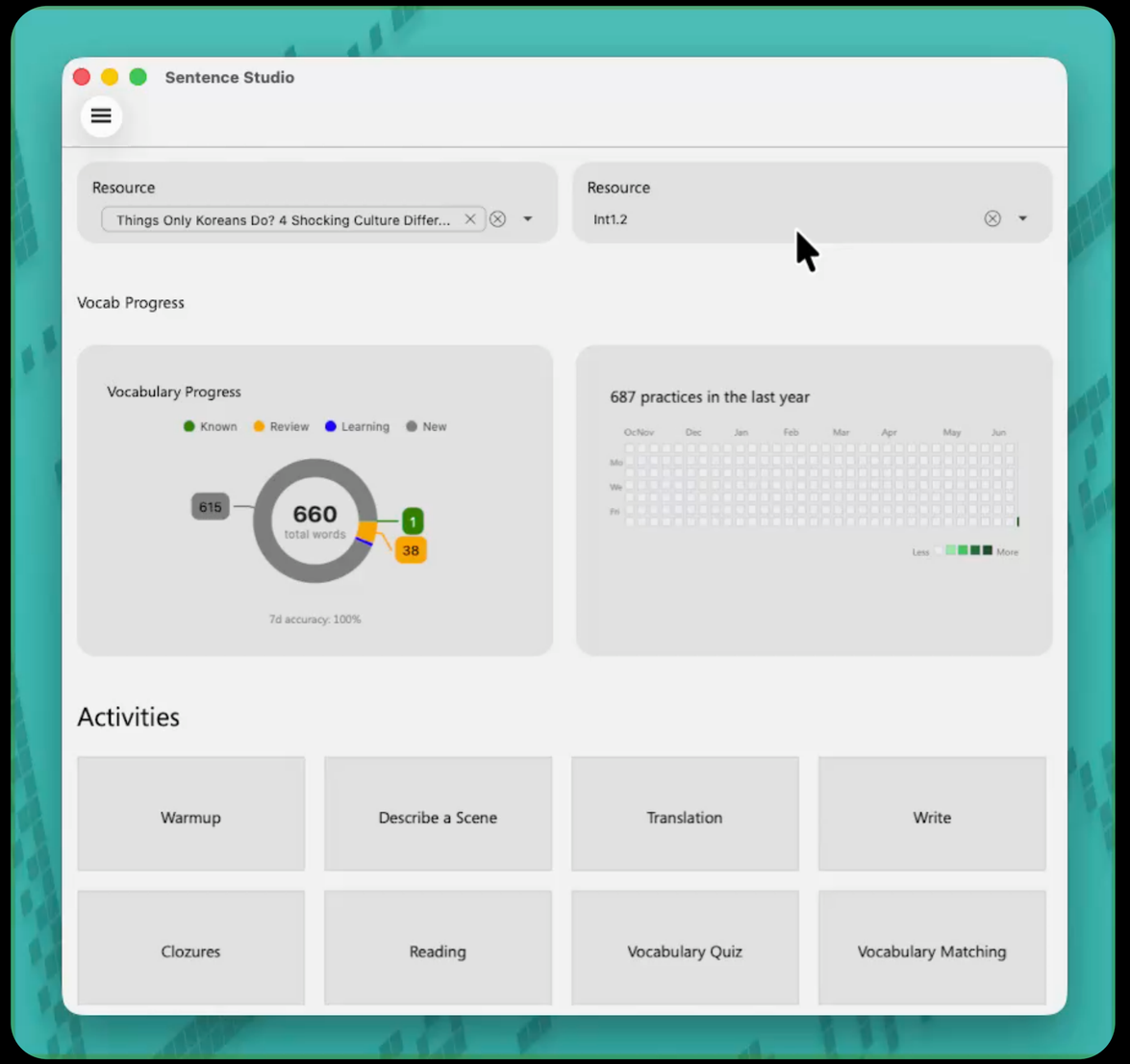
Cloud

ElevenLabs offers a large library of natural voice models, or train you own.

Sentence Studio



https://elevenlabs.io/



ElevenLabs TextToSpeech

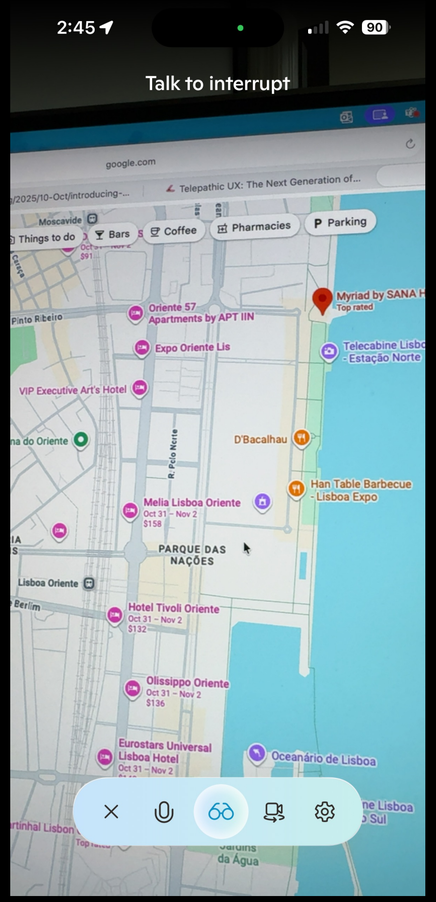
```
// Create audio generation options with latest model and context parameters
var request = new TextToSpeechRequest(
    voice,
    text,
    voiceSettings: new VoiceSettings(stability, similarityBoost) { Speed = speed },
    model: Model.MultiLingualV2,
    previousText: previousText,
    nextText: nextText); // Using latest multilingual model

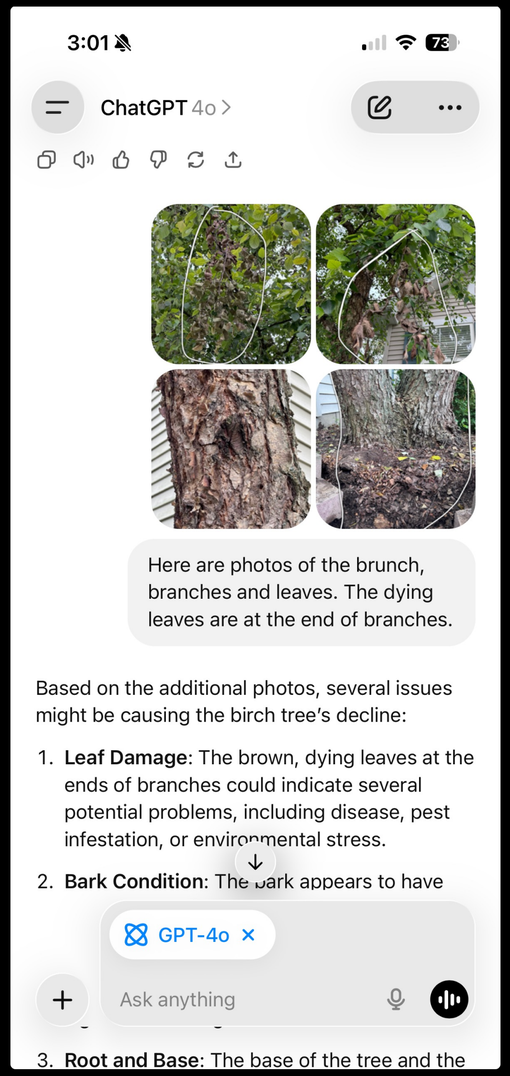
// Generate the speech using the proper API call
var audioBytes = await _client.TextToSpeechEndpoint.TextToSpeechAsync(
    request,
    cancellationToken: cancellationToken);

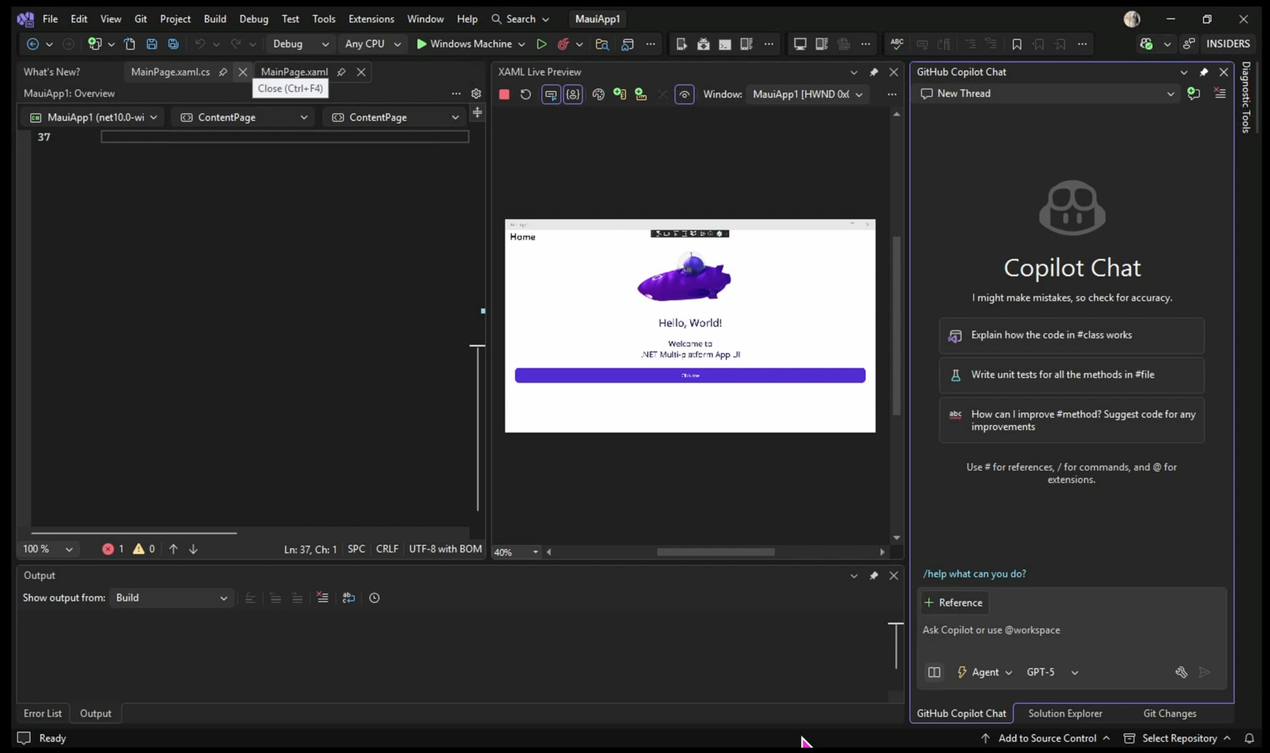

// Create a memory stream from the audio bytes
var audioStream = new MemoryStream(audioBytes.ClipData.ToArray());
```

https://github.com/RageAgainstThePixel/ElevenLabs-DotNet

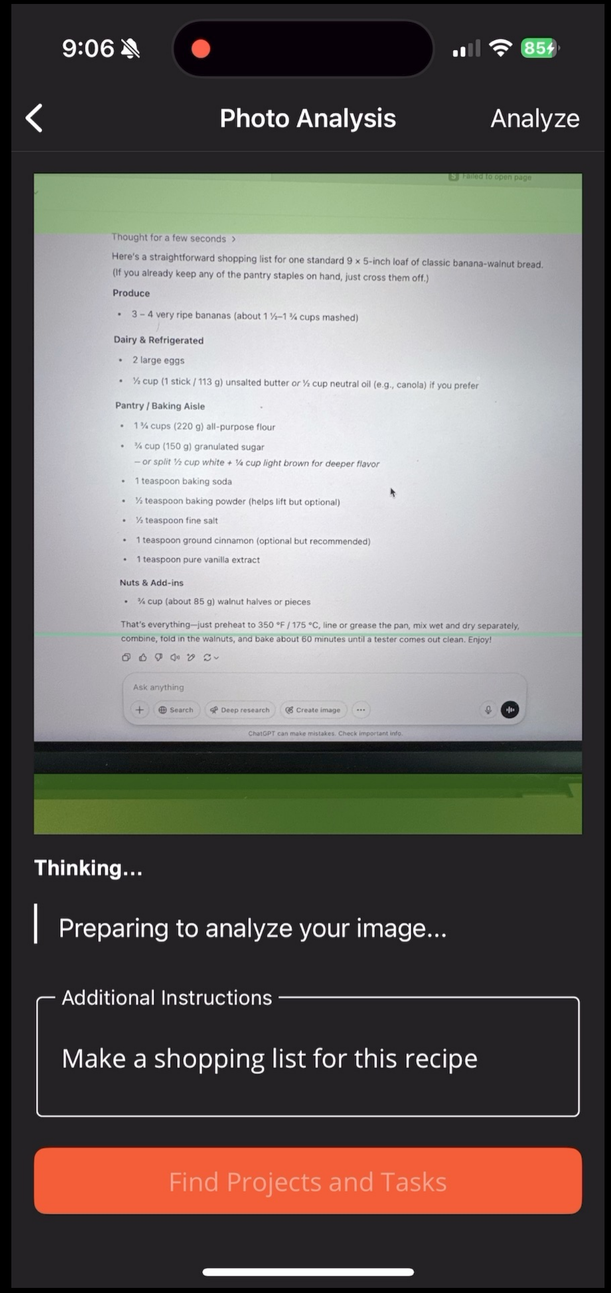
Vision
Multimodal







04.03 Vision demo



PhotoPageModel.cs – Getting a Photo

```
FileResult? result = null;
if (DeviceInfo.Idiom == DeviceIdiom.Desktop)
{
  result = await MediaPicker.PickPhotoAsync(new MediaPickerOptions
    {
      Title = "Select a photo”
    });
}
else
{
  if (!MediaPicker.IsCaptureSupported)
  ...

  result = await MediaPicker.CapturePhotoAsync(new MediaPickerOptions
    {
      Title = "Take a photo"
    });
}
```

PhotoPageModel.cs – Sending Photo and Prompt to AI

```
var client = _chatClientService.GetClient();

byte[] imageBytes = File.ReadAllBytes(ImagePath);

var msg = new Microsoft.Extensions.AI.ChatMessage(ChatRole.User,
[
  new TextContent(prompt.ToString()),
  new DataContent(imageBytes, mediaType: "image/png")
]);

var apiResponse = await client.GetResponseAsync<ProjectsJson>(msg);
```

05 UX Design Principles

Microsoft's AI Principles



Mitigation Layers



Design for responsible human-AI interaction

The HAX Toolkit is a set of practical tools to help create responsible and effective human-AI interactions, including guidelines, workbooks, design patterns, and a design library

Ground your model and steer its behavior

Recommended System Message Framework provides instructions, clear directives, background information relevant to the task, guidelines on tone and style of responses, and safety guidelines

Monitor and protect model inputs and outputs

Azure AI Content Safety detects and moderates harmful user-generated and AI-generated content in applications and services using text and image APIs

Choose the right model for your use case

Azure AI Foundry model catalog provides a broad selection of models and helpful tools to guide your choice

Resources for getting started

Microsoft's approach foundations

- 2024 Transparency Report
- Responsible AI Tools and Practices | Microsoft AI
- Responsible AI Standard v2
- Responsible AI Impact Assessment Template
- Azure AI to help you build more secure and trustworthy generative AI applications

Special topics

- Planning red teaming for large language models (LLMs)
- Microsoft's open automation framework to red team generative AI Systems
- Lessons from Red Teaming 100 Gen AI products whitepaper
- HAX Playbook for human-AI interaction design
- Customer Copyright Commitment Required Mitigations
- Accelerating Sustainability with AI: A Playbook

Learning modules

- Microsoft AI - Learning and Community Hub
- Embrace responsible AI principles and practices - Training
- Fundamentals of Responsible Generative AI – Training
- Responsible generative AI in AI Studio – Training
- AI Red Teaming Learning hub
- AI Literacy Starting Guide

Microsoft on the Issues

- How do we best govern AI?
- Our commitments to advance safe, secure, and trustworthy AI
- Microsoft announces new Copilot Copyright Commitment
- Combating AI deepfakes in elections through today's new tech accord
- Protecting the data of our commercial and public sector customers in the AI era
- Global Governance: Goals and Lessons for AI
- Innovating in line with the European Union's AI Act

Find resources like these and more at aka.ms/RAI

6 Design Principles for AI Applications

Design Responsibly

- Solve real user problems
- Minimize user harms
- Use human-centered design
- Expose or limit emergent behaviors
- Test & monitor for user harms

Design for Mental Models

- Orient the user to generative variability
- Teach effective use
- Understand the user's mental model
- Teach AI about the user

Design for Appropriate Trust & Reliance

- Help a user know when they should/should not rely on AI output
- Be clear on how well AI performs for a given task
- Provide rationales for outputs
- Use friction to prevent overreliance
- Encourage critical thinking
- Signify the role of the AI

6 Design Principles for AI Applications

Design for Generative Variability

- Leverage multiple outputs
- Visualize the user's journey
- Enable curation & annotation
- Draw attention to differences or variations across outputs

Design for Co-Creation

- Help the user craft effective outcome specifications
- Provide generic input parameters
- Provide controls relevant to the use case & technology
- Support co-editing of generated outputs

Design for Imperfection

- Make uncertainty visible
- Evaluate outputs for quality
- Offer ways to improve outputs
- Provide feedback mechanisms

06 Next Steps

https://github.com/davidortinau/telepathy

Thank you



David Ortinau | david.ortinau@microsoft.com
Principal Product Manager